

Military Asset Management System - Documentation

Project Overview

The **Military Asset Management System** is a secure, role-based platform designed for commanders and logistics personnel to track and manage critical assets (vehicles, weapons, and ammunition) across multiple military bases. It facilitates real-time inventory tracking, asset transfers, and assignment/expenditure logging.

Tech Stack & Architecture

- **Frontend:** React.js (Vite), React Router, Context API for state management, Lucide-React for iconography.
- **Backend:** Node.js, Express.js.
- **Database:** MongoDB (Mongoose ODM).
- **Authentication:** JWT (JSON Web Tokens) with bcryptjs for password hashing.
- **Styling:** Vanilla CSS with a modern, "Chocolate & Cream" premium aesthetic.

Data Models / Schema

User

- `username`: String (Unique)
- `password`: String (Hashed)
- `role`: String (Admin, Base Commander, Logistics Officer)
- `base`: String (Assigned base or 'All')

Asset

- `name`: String
- `category`: String (Vehicle, Weapon, Ammunition)
- `base`: String
- `quantity`: Number
- `status`: String (Available, In Use, Maintenance)
- `description`: String

Transfer

- `assetId`: ObjectId (Reference to Asset)
- `assetName`: String
- `fromBase`: String
- `toBase`: String
- `quantity`: Number
- `status`: String (Completed)
- `transferDate`: Date

Assignment

- `assetId`: ObjectId (Reference to Asset)
- `assetName`: String
- `assignedTo`: String
- `quantity`: Number
- `base`: String
- `type`: String (Assignment, Expenditure)
- `date`: Date

Role-Based Access Control (RBAC)

- **Admin:** Full access to all data, can manage users, assets, transfers, and assignments.
- **Logistics Officer:** Can record new purchases (Assets), initiate transfers, and view all data.
- **Base Commander:** Can view assets and transfers related to their specific base. Can record assignments and expenditures for their base.

API Logging

The system uses `morgan` middleware in the backend to log all incoming HTTP requests to the console in 'dev' format, providing real-time visibility into API interactions.

Setup Instructions

Backend

1. Navigate to the `backend` directory.
2. Install dependencies: `npm install`.
3. Start the server: `npm start`.
 - *Note: The server uses an in-memory MongoDB by default if no `MONGO_URI` is provided in `.env`.*

Frontend

1. Navigate to the `frontend` directory.
2. Install dependencies: `npm install`.
3. Start the development server: `npm run dev`.
4. Access the app at the URL provided in the terminal (usually `http://localhost:5173`).

API Endpoints

Auth

- POST /api/auth/login: Authenticate user and receive JWT.

Assets

- GET /api/assets: Fetch all assets (Filtered by base for Commanders).
- POST /api/assets: Create new asset (Purchases).
- PUT /api/assets/:id: Update asset details.

Transfers

- GET /api/transfers: Fetch transfer history.
- POST /api/transfers: Initiate a transfer between bases.

Assignments

- GET /api/assignments: Fetch assignment/expenditure records.
- POST /api/assignments: Record a new assignment or expenditure.
- DELETE /api/assignments/:id: Remove a record and return assets (if applicable).

Login Credentials

Role	Username	Password	Base
Admin	admin	admin123	All
Commander	commander_alpha	cmd123	Base Alpha
Commander	commander_bravo	cmd123	Base Bravo
Logistics	logistics	log123	All